

LAPORAN TUGAS BESAR 1
IF2224 Teori Bahasa Formal dan Automata
Semester II 2025/2026

Arion Compiler Milestone 1 - Lexical Analysis



Stevanus Agustaf Wongso	13524020
Renuno Yuqa Frinardi	13524080
Valentino Daniel Kusumo	13524104
Michael James Liman	13524106

Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

DAFTAR ISI

DAFTAR ISI.....	1
BAB I	
DASAR TEORI.....	2
1. Bahasa Pemrograman dan Proses Kompilasi.....	2
2. Lexical Analysis.....	2
3. Lexer.....	2
4. Token.....	2
5. Regular Expression (Regex).....	3
6. Finite Automata.....	3
7. Diagram Transisi DFA.....	3
BAB II	
PERANCANGAN DAN IMPLEMENTASI.....	4
1. Perancangan.....	4
2. Implementasi.....	17
2.1. Arsitektur File.....	17
2.2. Struktur Data.....	17
2.3. Keyword Mapping.....	18
2.4. Implementasi DFA.....	18
BAB III	
PENGUJIAN.....	21
1. Test Case 1 – Normal Case.....	21
2. Test Case 2 – Integer and Real cases.....	21
3. Test Case 3 – Comments and Case-Insensitive Cases.....	22
4. Test Case 4 – All Operators Cases.....	23
5. Test Case 5 – Unknown, Char, and String Cases.....	23
BAB IV	
KESIMPULAN DAN SARAN.....	24
1. Kesimpulan.....	24
2. Saran.....	24
LAMPIRAN.....	25
REFERENSI.....	26

BAB I

DASAR TEORI

1. Bahasa Pemrograman dan Proses Kompilasi

Bahasa pemrograman adalah sekumpulan instruksi, sintaksis, dan aturan yang digunakan manusia untuk berkomunikasi dengan komputer tanpa harus berhadapan langsung dengan bahasa mesin. Karena CPU hanya memahami bahasa mesin, setiap program yang ditulis dalam *high-level language* (umumnya sering digunakan) harus melalui serangkaian proses transformasi sebelum dapat dieksekusi. Proses transformasi inilah yang disebut kompilasi dan program yang melakukannya disebut *compiler* dan *interpreter*. Secara umum, *compiler/interpreter* bekerja dalam empat tahapan utama yang berjalan secara berurutan, yaitu **Lexical Analysis, Syntax Analysis, Semantic Analysis, dan Intermediate Code Generation**. Setiap tahapan mengambil hasil dari tahapan sebelumnya dan menghasilkan representasi program yang semakin mendekati bahasa mesin.

2. Lexical Analysis

Lexical Analysis merupakan tahapan pertama dalam proses kompilasi. Pada tahap ini, program sumber yang pada dasarnya hanyalah sebuah rangkaian karakter mentah yang dibaca satu per satu, kemudian dikelompokkan menjadi unsur bermakna yang disebut *token*. Proses ini sering juga disebut *tokenizing* atau *scanning*. Tujuan utama *lexical analysis* adalah menyederhanakan *input* untuk tahap berikutnya (*parsing*), sehingga tahap *parsing* tidak perlu berurusan dengan karakter satu per satu, melainkan langsung dengan sebuah unit yang sudah terstruktur. Selain itu, pada tahap ini karakter-karakter yang tidak penting bagi proses kompilasi, seperti spasi, baris baru, tab, dan komentar juga dibuang atau diabaikan.

3. Lexer

Lexer membaca karakter *input* satu per satu dari kiri ke kanan, lalu mencocokkannya dengan pola-pola yang telah didefinisikan untuk setiap jenis token. Begitu suatu urutan karakter cocok dengan salah satu pola, *lexer* akan membuat sebuah *token* yang merepresentasikan urutan tersebut, kemudian mereset statusnya dan melanjutkan pembacaan karakter berikutnya. Jika *lexer* menemukan urutan karakter yang tidak cocok dengan pola mana pun, ia harus mampu melaporkan kesalahan secara informatif tanpa langsung menghentikan keseluruhan proses.

4. Token

Token adalah unit terkecil yang memiliki makna semantik dalam sebuah bahasa pemrograman. Setiap *token* terdiri dari dua komponen utama, yaitu jenis yang menggolongkan token ke dalam kategori tertentu, dan nilai yang menyimpan representasi literal dari token tersebut pada *source code*. Jenis-jenis token yang umumnya ditemui dalam bahasa pemrograman adalah **keyword**, misalnya *begin*, *end*, *if*, *while*, **identifier** (nama yang didefinisikan untuk variabel, fungsi, atau prosedur), **konstanta** (nilai tetap seperti bilangan bulat, bilangan riil, atau karakter),

operator (simbol yang merepresentasikan operasi aritmatika, relasional, atau logika), serta **delimiter** (tanda baca pemisah seperti titik koma, koma, dan tanda kurung).

5. Regular Expression (Regex)

Regex adalah deretan karakter khusus yang membentuk pola pencarian untuk mencocokkan, mencari, mengganti, atau memanipulasi teks/string. Regex adalah alat yang sangat efisien untuk validasi data dan pengolahan teks kompleks.

6. Finite Automata

Finite Automata adalah model komputasi abstrak yang digunakan untuk mengenali pola-pola dalam string. Finite Automata secara formal didefinisikan sebagai 5-tuple

$$M = (Q, \Sigma, \delta, q_0, F)$$

- Q (Himpunan State): Kumpulan state yang mungkin ada dalam mesin tersebut
- Σ (Alfabet): Himpunan terbatas dari simbol-simbol *input* yang diterima oleh mesin
- δ (Fungsi Transisi): Aturan yang menentukan perpindahan dari satu state ke state lainnya berdasarkan *input* yang diberikan
- q_0 (State Awal): State yang menjadi titik mulai saat mesin pertama kali dijalankan
- F (Himpunan State Akhir): Himpunan bagian dari Q yang berisi state-state penerima. Jika proses berakhir di salah satu state ini, maka string *input* dianggap diterima

7. Diagram Transisi DFA

Diagram transisi adalah representasi grafis dari DFA berupa graf berarah, di mana setiap node (simpul) merepresentasikan sebuah state, setiap *edge* berlabel merepresentasikan fungsi transisi dari satu state ke state lain berdasarkan simbol *input* tertentu, state awal ditandai dengan panah masuk dan state penerima (*accepting state*) ditandai dengan lingkaran ganda.

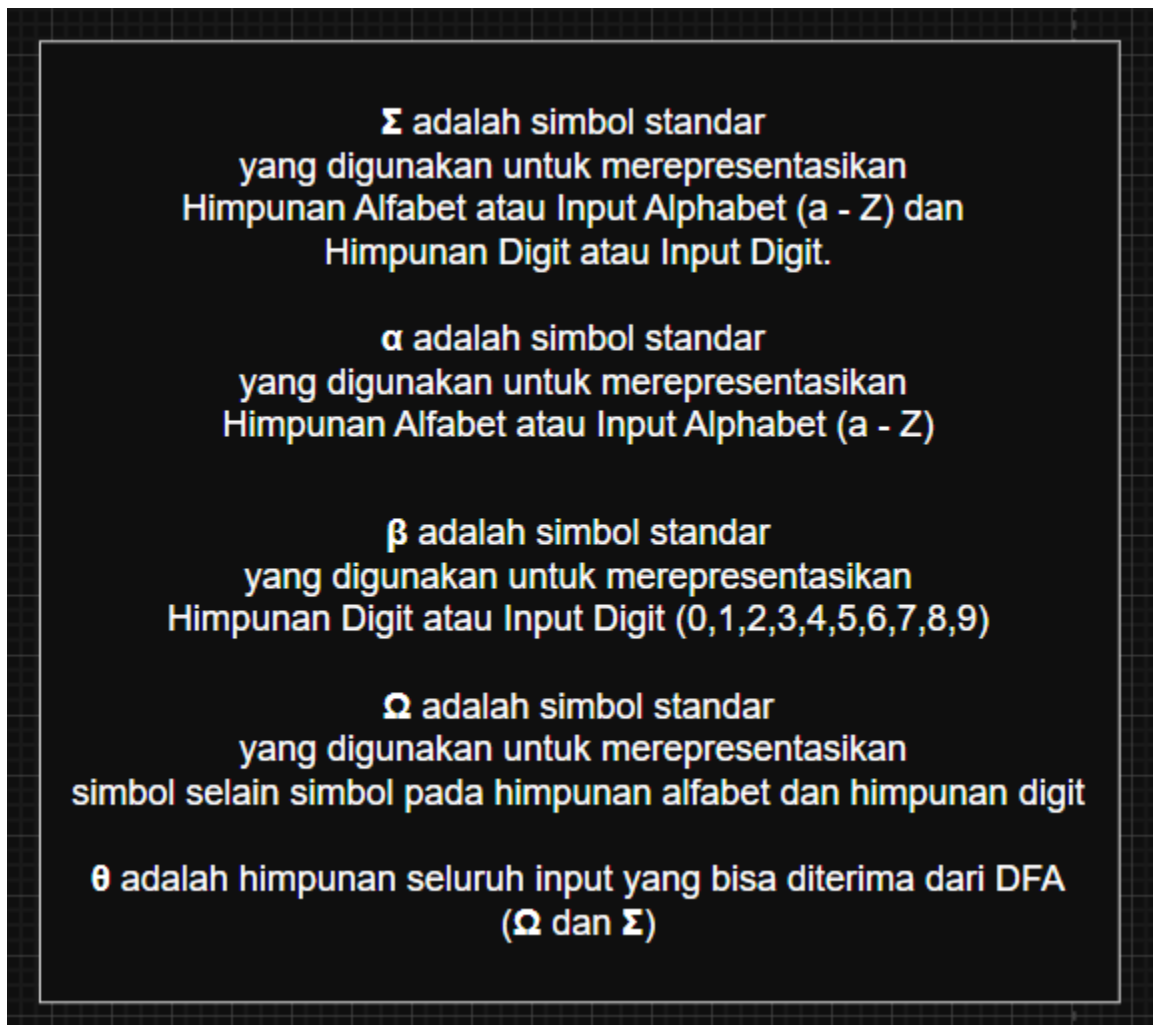
BAB II

PERANCANGAN DAN IMPLEMENTASI

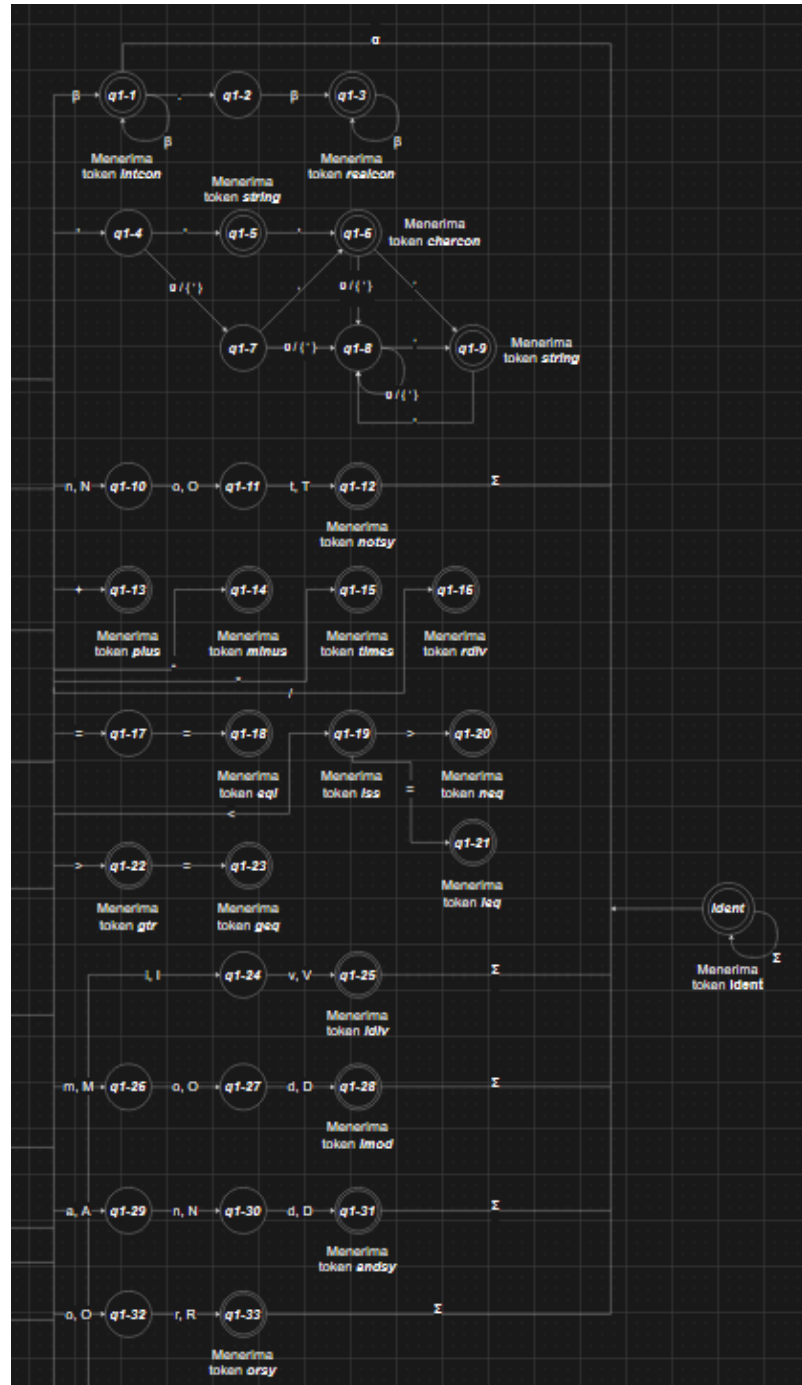
1. Perancangan

Diagram Transisi DFA dibuat menggunakan draw.io. Nomor *state* yang digunakan hanya untuk memudahkan perancangan diagram transisi DFA.

Sebelum masuk ke *state*, ada beberapa simbol yang didefinisikan untuk memudahkan perancangan DFA, yaitu Σ yang berarti *input* (Alphabet a-z dan Digit), α yang berarti *input* Alphabet, β yang berarti *input* Digit, Ω yang berarti simbol yang merepresentasikan *input* selain Alphabet dan Digit, dan θ yang berarti semua *input* (Ω , β , dan Σ). **q0** sebagai **start state** (cabang utama).



Gambar 1: Gambar Definisi Simbol



Gambar 2: Cupilkan Diagram Transisi DFA (State q1-1 sampai q1-33)

State - state yang terdefinisi :

- **q1-1:** Menerima seluruh simbol dalam bentuk digit (0...9) (Menerima token *intcon*)
- **q1-2:** Menerima simbol titik setelah membaca simbol digit sebelumnya
- **q1-3:** Menerima seluruh simbol dalam bentuk bilangan real(0...9) (Menerima token *realcon*)

- **q1-4:** Menerima simbol petik satu
- **q1-5:** Menerima string kosong (Menerima token **string**)
- **q1-6:** Menerima sebuah token char yang terdefinisi (Menerima token **charcon**)
- **q1-7:** Menerima sebuah simbol selain simbol petik
- **q1-8:** Menerima sebuah simbol selain simbol petik setelah menerima sebuah simbol lain sebelumnya yang diawali simbol petik di awal pembacaan
- **q1-9:** Menerima sebuah string yang terdefinisi (Menerima token **string**)
- **q1-10:** Menerima sebuah input n atau N
- **q1-11:** Menerima sebuah input o atau O setelah membaca n atau N
- **q1-12:** Menerima sebuah token not (Menerima token **notsy**)
- **q1-13:** Menerima sebuah token tambah (Menerima token **plus**)
- **q1-14:** Menerima sebuah token minus (Menerima token **minus**)
- **q1-15:** Menerima sebuah token kali (Menerima token **times**)
- **q1-16:** Menerima sebuah token bagi bilangan real (Menerima token **rdiv**)
- **q1-17:** Menerima sebuah simbol sama dengan
- **q1-18:** Menerima sebuah simbol sama dengan setelah membaca sama dengan (Menerima token **eqI**)
- **q1-19:** Menerima sebuah simbol kurang dari (Menerima token **lss**)
- **q1-20:** Menerima sebuah simbol lebih dari setelah membaca simbol kurang dari (Menerima token **neq**)
- **q1-21:** Menerima sebuah simbol sama dengan setelah membaca simbol kurang dari (Menerima token **leq**)
- **q1-22:** Menerima sebuah simbol lebih dari (Menerima token **gtr**)
- **q1-23:** Menerima sebuah sama dengan setelah membaca simbol lebih dari (Menerima token **geq**)
- **q1-24:** Menerima sebuah input i atau I setelah membaca d atau D
- **q1-25:** Menerima sebuah token pembagian integer (Menerima token **idiv**)
- **q1-26:** Menerima sebuah input m atau M
- **q1-27:** Menerima sebuah input o atau O setelah membaca m atau M
- **q1-28:** Menerima sebuah token modulo (Menerima token **imod**)
- **q1-29:** Menerima sebuah input a atau A
- **q1-30:** Menerima sebuah input n atau N setelah membaca a atau A
- **q1-31:** Menerima sebuah token and (Menerima token **andsy**)
- **q1-32:** Menerima sebuah input o atau O
- **q1-33:** Menerima sebuah token or (Menerima token **orsy**)
- **ident:** Menerima sebuah token nama (Menerima token **ident**)

Pada gambar di bagian state dengan awalan q1,, DFA bekerja dengan cara mengenali sebuah token sesuai dengan urutan kemunculan dari simbolnya. Karena token masih sederhana, seluruh simbol akan mengecek secara rigid urutan sebuah token yang terdefinisi, dan apabila terdapat input yang salah, maka akan langsung berubah ke state input invalid (*tidak digambarkan karena akan menimbulkan arrow yang berantakan*). Sistem pembacaan hanya terlihat kompleks pada bagian **string** karena harus mengurus case bagi string kosong dan kemunculan tanda petik satu dalam input.



Gambar 3: Cupilkan Diagram Transisi DFA (State q2-1 sampai q2-11)

State - state yang terdefinisi :

- **Newline:** Menerima input karakter baris baru
- **Spasi:** Menerima input karakter spasi
- **q2-1:** Menerima simbol ((Menerima token **lparent**)
- **q2-2:** Menerima simbol * dan telah membaca (sebelumnya
- **q2-3:** Menerima semua *input* kecuali) dan *

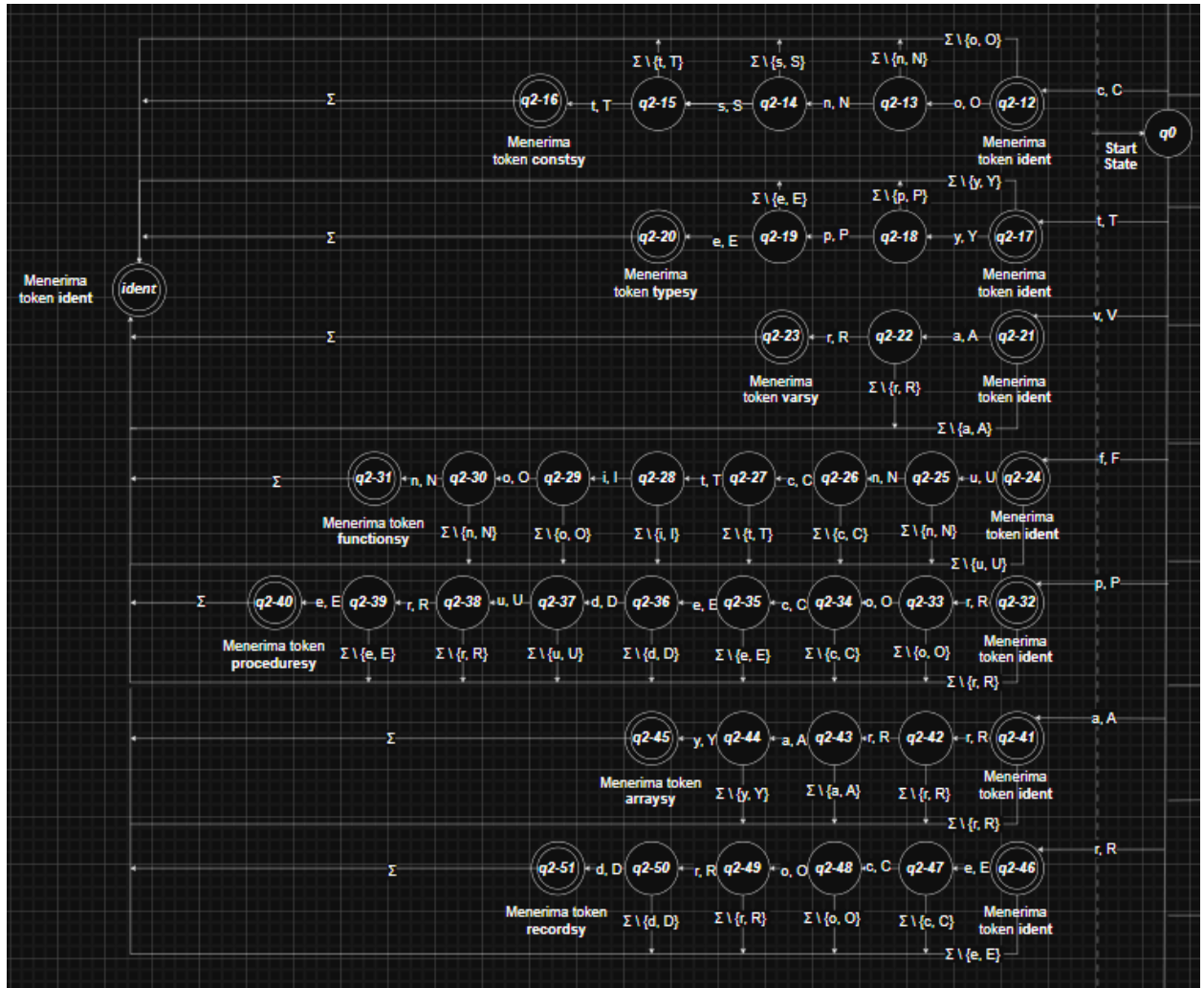
- **q2-3a:** Menerima input * dan telah membaca * sebelumnya (minimal 2 *)
- **q2-4:** Menerima simbol) (Menerima token **rparent**)
- **q2-5:** Menerima simbol [(Menerima token **lbrack**)
- **q2-6:** Menerima simbol] (Menerima token **rbrack**)
- **q2-7:** Menerima simbol , (Menerima token **comma**)
- **q2-8:** Menerima simbol ; (Menerima token **semicolon**)
- **q2-9:** Menerima simbol . (Menerima token **period**)
- **q2-10:** Menerima simbol : (Menerima token **colon**)
- **q2-11:** Menerima simbol = setelah membaca : (Menerima token **becomes**)

Proses mengenali token **comment** (*khusus format (*.*)*) dimulai dari state **q2-1** setelah membaca (. Jika karakter berikutnya adalah *, maka masuk ke **q2-2**. Jika input setelah (bukan *, sistem langsung menghasilkan **Error**.

Di dalam **q2-2**, jika ditemukan * lagi (*potensi penutup*), proses berpindah ke **q2-3a**. Jika inputnya adalah) (*kurung tutup tanpa bintang sebelumnya*), sistem menghasilkan Error. Namun, jika inputnya adalah karakter lain, proses masuk ke **q2-3** untuk membaca isi komentar.

Pada **q2-3** (Isi Komentar), sistem akan terus melakukan looping membaca semua karakter sampai menemukan *, yang kemudian akan mengarahkan kembali ke **q2-3a**. Di **q2-3a**, jika ditemukan), maka token **Comment** dinyatakan valid. Jika ditemukan karakter lain, proses kembali ke q2-3 untuk mencari bintang penutup berikutnya.

Untuk seluruh state di luar rangkaian **q2-2 hingga q2-3a**, DFA bekerja dengan prinsip pengenalan langsung. Pada jalur ini, sistem tidak memerlukan verifikasi karakter ganda, begitu input yang bersesuaian diterima, DFA akan langsung berpindah ke **final state** terkait dan mengenali token tersebut secara instan.



Gambar 4: Cupilkan Diagram Transisi DFA (State q2-12 sampai q2-51)

State - state yang terdefinisi (*Karakter insensitive*) :

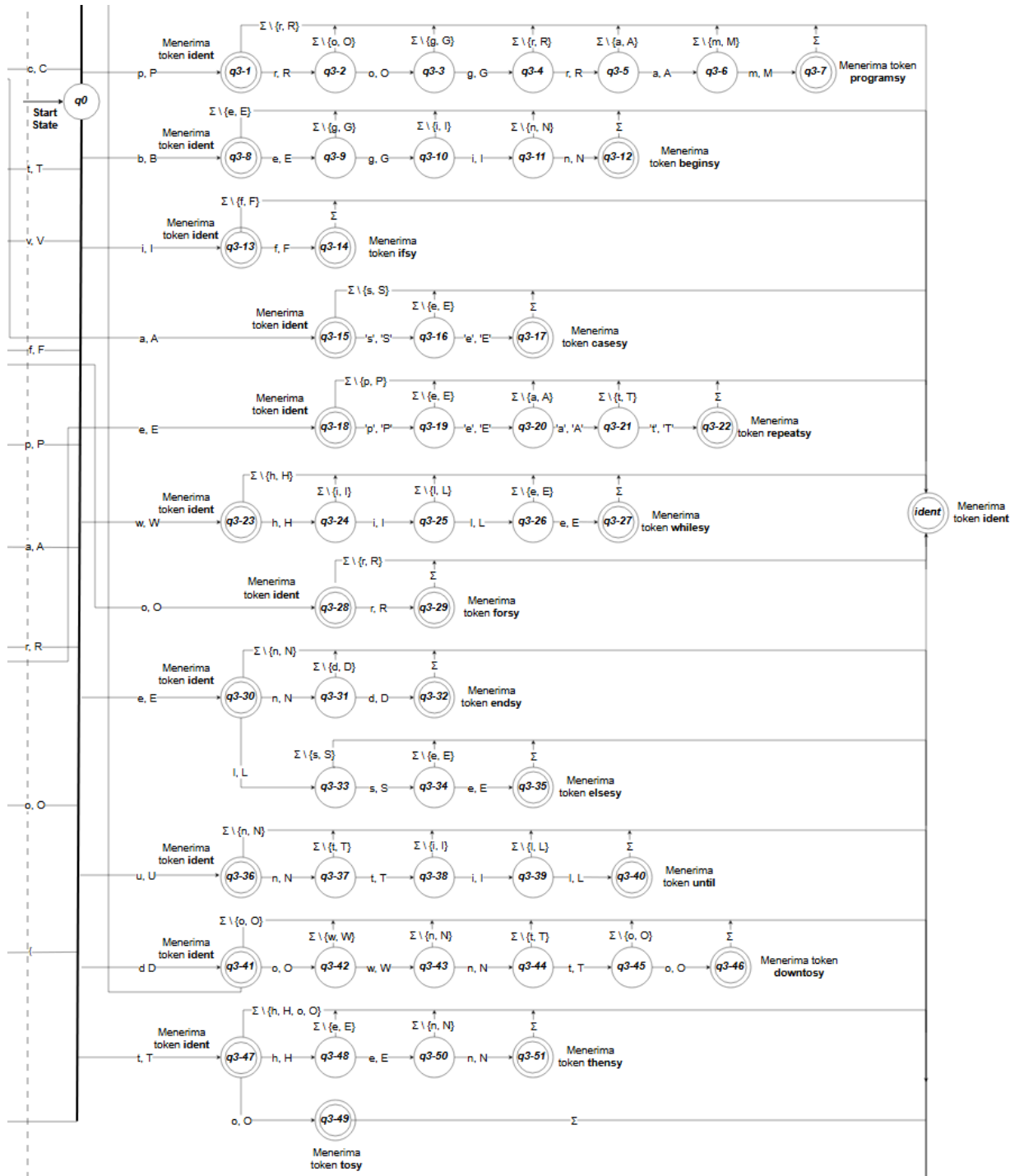
- **ident**: State akhir yang menerima token ident. State ini menampung semua kata yang tidak memenuhi kriteria kata kunci / variabel
- **q2-12**: Menerima karakter c, sebagai bagian dari pengecekan kata kunci pertama **const**
- **q2-13**: Menerima karakter o setelah c, sebagai bagian dari pengecekan kata kunci kedua **const**
- **q2-14**: Menerima karakter n setelah co, sebagai bagian dari pengecekan kata kunci ketiga **const**
- **q2-15**: Menerima karakter s setelah con, sebagai bagian dari pengecekan kata kunci keempat **const**
- **q2-16**: Menerima karakter t setelah urutan cons (**Menerima token constsy**)
- **q2-17**: Menerima karakter t, sebagai bagian dari pengecekan kata kunci pertama **type**
- **q2-18**: Menerima karakter y setelah t, sebagai bagian dari pengecekan kata kunci kedua **type**

- **q2-19:** Menerima karakter p setelah ty, sebagai bagian dari pengecekan kata kunci ketiga **type**
- **q2-20:** Menerima karakter e setelah urutan typ (**Menerima token typesy**).
- **q2-21:** Menerima karakter v, sebagai bagian dari pengecekan kata kunci pertama **var**
- **q2-22:** Menerima karakter a setelah v, sebagai bagian dari pengecekan kata kunci kedua **var**
- **q2-23:** Menerima karakter r setelah urutan va (**Menerima token varsy**)
- **q2-24:** Menerima karakter f, sebagai bagian dari pengecekan kata kunci pertama **function**
- **q2-25:** Menerima karakter u setelah f, sebagai bagian dari pengecekan kata kunci kedua **function**
- **q2-26:** Menerima karakter n setelah fu, sebagai bagian dari pengecekan kata kunci ketiga **function**
- **q2-27:** Menerima karakter c setelah fun, sebagai bagian dari pengecekan kata kunci keempat **function**
- **q2-28:** Menerima karakter t setelah func, sebagai bagian dari pengecekan kata kunci kelima **function**
- **q2-29:** Menerima karakter i setelah funct, sebagai bagian dari pengecekan kata kunci keenam **function**
- **q2-30:** Menerima karakter o setelah functi, sebagai bagian dari pengecekan kata kunci ketujuh **function**
- **q2-31:** Menerima karakter n setelah urutan functio (**Menerima token functionsy**)
- **q2-32:** Menerima karakter p, sebagai bagian dari pengecekan kata kunci pertama **procedure**
- **q2-33:** Menerima karakter r setelah p, sebagai bagian dari pengecekan kata kunci kedua **procedure**
- **q2-34:** Menerima karakter o setelah pr, sebagai bagian dari pengecekan kata kunci ketiga **procedure**
- **q2-35:** Menerima karakter c setelah pro, sebagai bagian dari pengecekan kata kunci keempat **procedure**
- **q2-36:** Menerima karakter e setelah proc, sebagai bagian dari pengecekan kata kunci kelima **procedure**
- **q2-37:** Menerima karakter d setelah proce, sebagai bagian dari pengecekan kata kunci keenam **procedure**
- **q2-38:** Menerima karakter u setelah proced, sebagai bagian dari pengecekan kata kunci ketujuh **procedure**
- **q2-39:** Menerima karakter r setelah procedu, sebagai bagian dari pengecekan kata kunci kedelapan **procedure**
- **q2-40:** Menerima karakter e setelah urutan procedur (**Menerima token proceduresy**)
- **q2-41:** Menerima karakter a, sebagai bagian dari pengecekan kata kunci pertama **array**
- **q2-42:** Menerima karakter r setelah a, sebagai bagian dari pengecekan kata kunci kedua **array**

- **q2-43:** Menerima karakter r setelah ar, sebagai bagian dari pengecekan kata kunci ketiga **array**
- **q2-44:** Menerima karakter a setelah arr, sebagai bagian dari pengecekan kata kunci keempat **array**
- **q2-45:** Menerima karakter y setelah urutan arra (**Menerima token arraysy**)
- **q2-46:** Menerima karakter r, sebagai bagian dari pengecekan kata kunci pertama **record**
- **q2-47:** Menerima karakter e setelah r, sebagai bagian dari pengecekan kata kunci kedua **record**
- **q2-48:** Menerima karakter c setelah re, sebagai bagian dari pengecekan kata kunci ketiga **record**
- **q2-49:** Menerima karakter o setelah rec, sebagai bagian dari pengecekan kata kunci keempat **record**
- **q2-50:** Menerima karakter r setelah reco, sebagai bagian dari pengecekan kata kunci kelima **record**
- **q2-51:** Menerima karakter d setelah urutan recor (**Menerima token recordsy**)

Sistem identifikasi token ini bekerja dengan membedakan antara *Reserved Words* (seperti *function*, *array*, *dsb*) dan *Ident* melalui beberapa proses berikut:

1. Validasi Bertahap: Setiap kata kunci memiliki jalur transisi linear. Jika urutan karakter yang dimasukkan tepat (misalnya p-r-o-c...), maka DFA akan berpindah ke *state* berikutnya sesuai urutan "kata kunci ke-n" hingga mencapai **final state**.
2. Proses *fallback*: Jika di tengah jalan karakter yang terbaca tidak sesuai dengan ejaan kata kunci (misalnya mengetik *process* yang berbeda dengan *procedure*), DFA akan melakukan transisi dari jalur utama menuju *state ident*.
3. Klasifikasi Ident: Di dalam *state ident*, semua input yang berbeda atau tidak terdaftar sebagai kata kunci akan dikategorikan sebagai token **ident**. Hal ini menjamin fleksibilitas dalam penamaan variabel tanpa mengganggu fungsi kata kunci utama.



Gambar 5: Cupilkan Diagram Transisi DFA (State q3-1 sampai q3-51)

Pengecekan kata kunci program

- q3-1: Menerima karakter p atau P, sebagai bagian dari pengecekan kata kunci pertama program

- q3-2: Menerima karakter r atau R setelah p, sebagai bagian dari pengecekan kata kunci kedua program
- q3-3: Menerima karakter o atau O setelah pr, sebagai bagian dari pengecekan kata kunci ketiga program
- q3-4: Menerima karakter g atau G setelah pro, sebagai bagian dari pengecekan kata kunci keempat program
- q3-5: Menerima karakter r atau R setelah prog, sebagai bagian dari pengecekan kata kunci kelima program
- q3-6: Menerima karakter a atau A setelah progr, sebagai bagian dari pengecekan kata kunci keenam program
- q3-7: Menerima karakter m atau M setelah progra (Menerima token programsy)

Pengecekan kata kunci **begin**

- q3-8: Menerima karakter b atau B, sebagai bagian dari pengecekan kata kunci pertama begin
- q3-9: Menerima karakter e atau E setelah b, sebagai bagian dari pengecekan kata kunci kedua begin
- q3-10: Menerima karakter g atau G setelah be, sebagai bagian dari pengecekan kata kunci ketiga begin
- q3-11: Menerima karakter i atau I setelah beg, sebagai bagian dari pengecekan kata kunci keempat begin
- q3-12: Menerima karakter n atau N setelah begi (Menerima token beginsy)

Pengecekan kata kunci **if**

- q3-13: Menerima karakter i atau I, sebagai bagian dari pengecekan kata kunci pertama if
- q3-14: Menerima karakter f atau F setelah i (Menerima token ifsy)

Pengecekan kata kunci **case**

- q3-15: Menerima karakter c atau C, sebagai bagian dari pengecekan kata kunci pertama case
- q3-16: Menerima karakter a atau A setelah c, sebagai bagian dari pengecekan kata kunci kedua case
- q3-17: Menerima karakter s atau S setelah ca, sebagai bagian dari pengecekan kata kunci ketiga case
- q3-18: Menerima karakter e atau E setelah cas (Menerima token casesy)

Pengecekan kata kunci **repeat**

- q3-19: Menerima karakter r atau R, sebagai bagian dari pengecekan kata kunci pertama repeat
- q3-20: Menerima karakter e atau E setelah r, sebagai bagian dari pengecekan kata kunci kedua repeat

- q3-21: Menerima karakter p atau P setelah re, sebagai bagian dari pengecekan kata kunci ketiga repeat
- q3-22: Menerima karakter e atau E setelah rep, sebagai bagian dari pengecekan kata kunci keempat repeat
- q3-23: Menerima karakter a atau A setelah repe, sebagai bagian dari pengecekan kata kunci kelima repeat
- q3-24: Menerima karakter t atau T setelah repea (Menerima token repeatsy)

Pengecekan kata kunci while

- q3-25: Menerima karakter w atau W, sebagai bagian dari pengecekan kata kunci pertama while
- q3-26: Menerima karakter h atau H setelah w, sebagai bagian dari pengecekan kata kunci kedua while
- q3-27: Menerima karakter i atau I setelah wh, sebagai bagian dari pengecekan kata kunci ketiga while
- q3-28: Menerima karakter l atau L setelah whi, sebagai bagian dari pengecekan kata kunci keempat while
- q3-29: Menerima karakter e atau E setelah whil (Menerima token whilesy)

Pengecekan kata kunci for

- q3-30: Menerima karakter f atau F, sebagai bagian dari pengecekan kata kunci pertama for
- q3-31: Menerima karakter o atau O setelah f, sebagai bagian dari pengecekan kata kunci kedua for
- q3-32: Menerima karakter r atau R setelah fo (Menerima token forsy)

Pengecekan kata kunci end & else (berbagi awalan huruf 'e')

- q3-33: Menerima karakter e atau E, sebagai bagian dari pengecekan kata kunci pertama untuk end atau else
- q3-34: Menerima karakter n atau N setelah e, sebagai bagian dari pengecekan kata kunci kedua end
- q3-35: Menerima karakter d atau D setelah en (Menerima token endsy)
- q3-36: Menerima karakter l atau L setelah e (dari q3-33), sebagai bagian dari pengecekan kata kunci kedua else
- q3-37: Menerima karakter s atau S setelah el, sebagai bagian dari pengecekan kata kunci ketiga else
- q3-38: Menerima karakter e atau E setelah els (Menerima token else)

Pengecekan kata kunci until

- q3-39: Menerima karakter u atau U, sebagai bagian dari pengecekan kata kunci pertama until
- q3-40: Menerima karakter n atau N setelah u, sebagai bagian dari pengecekan kata kunci kedua until

- q3-41: Menerima karakter t atau T setelah un, sebagai bagian dari pengecekan kata kunci ketiga until
- q3-42: Menerima karakter i atau I setelah unt, sebagai bagian dari pengecekan kata kunci keempat until
- q3-43: Menerima karakter l atau L setelah unti (Menerima token untilsy)

Pengecekan kata kunci **downto**

- q3-44: Menerima karakter d atau D, sebagai bagian dari pengecekan kata kunci pertama downto
- q3-45: Menerima karakter o atau O setelah d, sebagai bagian dari pengecekan kata kunci kedua downto
- q3-46: Menerima karakter w atau W setelah do, sebagai bagian dari pengecekan kata kunci ketiga downto
- q3-47: Menerima karakter n atau N setelah dow, sebagai bagian dari pengecekan kata kunci keempat downto
- q3-48: Menerima karakter t atau T setelah down, sebagai bagian dari pengecekan kata kunci kelima downto
- q3-49: Menerima karakter o atau O setelah downt (Menerima token downtosy)

Pengecekan kata kunci **then & to** (berbagi awalan huruf 't')

- q3-50: Menerima karakter t atau T, sebagai bagian dari pengecekan kata kunci pertama untuk then atau to
- q3-51: Menerima karakter h atau H setelah t, sebagai bagian dari pengecekan kata kunci kedua then
- q3-52: Menerima karakter o atau O setelah t (dari q3-50) (Menerima token to)
- q3-53: Menerima karakter e atau E setelah th, sebagai bagian dari pengecekan kata kunci ketiga then
- q3-54: Menerima karakter n atau N setelah the (Menerima token thensy)

- Jika karakter selanjutnya adalah sembarang karakter selain * dan } (disimbolkan dengan $\emptyset \setminus \{*, \}\}$), mesin beralih ke state q3-58 untuk mulai membaca isi teks komentar.
 - Jika karakter *, langsung ke q3-59
- q3-58: State ini berfungsi untuk membaca isi (body) dari komentar.
 - Mesin akan terus berada di state ini (looping) selama ia membaca karakter apapun selain * dan }.
 - Jika ia membaca karakter }, maka komentar dianggap selesai dan mesin beralih ke Final State Comment.
 - Jika ia membaca karakter *, mesin beralih ke state q2-3 untuk mengecek kemungkinan penutup komentar dengan format lain.
- q3-59: State transisi untuk mengecek karakter setelah mesin menemukan tanda bintang (*) di dalam komentar.
 - Jika karakter selanjutnya adalah) (membentuk pola penutup *)), maka komentar dianggap selesai dan mesin beralih ke Final State Comment.
 - Jika karakter selanjutnya bukan penutup (disimbolkan dengan panah kembali $\emptyset \setminus \{ \}$), mesin menyimpulkan bahwa * tersebut hanyalah teks biasa di dalam komentar dan kembali ke state q2-2 (Tengah) untuk melanjutkan pembacaan isi komentar.
 - Jika karakter selanjutnya *, tetap di state tersebut.
- State Comment (Lingkaran Ganda): Merupakan Final State (State Akhir). Jika alur mencapai state ini, berarti mesin telah sukses memvalidasi pembukaan dan penutupan komentar, dan secara resmi Menerima token comment.

2. Implementasi

2.1. Arsitektur File

File	Peran
src/std.hpp	Aggregator STL (string, vector, map, dll.)
src/main.hpp	Aggregator header
src/main.cpp	Entry point: menerima nama file, memanggil DFAFileReader → Tokenizing → TokenPrinter
src/lexical/token_processing.hpp	Mendefinisikan struct Token dan mendeklarasikan fungsi Tokenizing()
src/lexical/token_processing.cpp	Implementasi utama DFA dengan 15 state untuk tokenisasi input
src/helpers/file_helper.cpp	Fungsi pendukung: membaca file, mencetak token, menampilkan pesan error

2.2. Struktur Data

```
struct Token {  
    string type;    // Tipe token  
    string value;   // Nilai literal token  
};
```

Menggunakan ADT Token untuk mendefinisikan token yang tersimpan.

2.3. Keyword Mapping

```
map<string,string> keyword = {  
    {"NOT", "notsy"},  
    {"DIV", "idiv"},  
    {"MOD", "imod"},  
    {"AND", "andsy"},  
    {"OR", "orsy"},  
    {"CONST", "constsy"},  
    {"TYPE", "typesy"},  
    {"VAR", "varsy"},  
    {"FUNCTION", "functionsy"},  
    {"PROCEDURE", "proceduresy"},  
    {"ARRAY", "arraysy"},  
    {"RECORD", "recordsy"},  
    {"PROGRAM", "programsy"},  
    {"BEGIN", "beginsy"},  
    {"IF", "ifsy"},  
    {"CASE", "casesy"},  
    {"REPEAT", "repeatsy"},  
    {"WHILE", "whilesy"},  
    {"FOR", "forsy"},  
    {"END", "endsy"},  
    {"ELSE", "elsesy"},  
    {"UNTIL", "untilsy"},  
    {"OF", "ofsy"},  
    {"DO", "dosy"},  
    {"TO", "tosy"},  
    {"DOWNTON", "downtosy"},  
    {"THEN", "thensy"},  
};
```

Mapping Keyword untuk menentukan token

Contoh penggunaan :

```
if (keyword.count(upper)) {  
    tokens.push_back({keyword[upper], ""});  
}  
else{  
    tokens.push_back({"ident",currWord});  
}
```

2.4. Implementasi DFA

DFA diimplementasikan menggunakan **enum class State** dan loop while (idx <= size).

Berikut implementasi **enum class state** :

```
enum class State{
    IDLE,
    NUM,
    REAL,
    STRING,
    ESCAPE,
    WORD,
    EQ,
    LE,
    GTR,
    COLON,
    CURLCMNT,
    COMMENTS,
    OPCMT,
    CLSCMT,
    NEGATIVE,
};
```

Pendefinisian State :

State	Keterangan
IDLE	State awal; menentukan token berikutnya dari karakter pertama
WORD	Mengakumulasi huruf/digit untuk keyword atau identifier
NUM	Mengakumulasi digit untuk bilangan bulat
REAL	Mengakumulasi digit setelah titik desimal
NEGATIVE	Menentukan apakah - adalah tanda minus atau negatif bilangan
STRING	Mengakumulasi karakter di dalam literal string/char
ESCAPE	Setelah menemukan penutup ', menentukan escape atau akhir literal
EQ	Setelah =, menentukan apakah == (eq)
LE	Setelah <, menentukan <>, <=, atau <
GTR	Setelah >, menentukan >= atau >
COLON	Setelah :, menentukan := atau :
OPCMT	Setelah (, menentukan komentar (* atau lparent
CURLCMNT	Di dalam komentar { }
COMMENTS	Di dalam komentar (* *)
CLSCMT	Setelah *, menentukan apakah penutup komentar *)

Cara Kerja DFA :

1. **IDLE** Membaca karakter pertama untuk menentukan state berikutnya. Spasi dilewati. Digit → NUM, huruf → WORD, ' (petik1) → STRING, operator/symbol → emit langsung atau masuk state lookahead terkait.
2. **WORD** — Mengakumulasi karakter alfanumerik. Selesai saat menemukan karakter lain: currWord dikonversi ke uppercase lalu dicek ke map keyword. Jika cocok → emit token keyword, jika tidak → emit ident.
3. **NUM** — Mengakumulasi digit. Jika menemukan . (titik) → pindah ke REAL. Jika tidak → emit intcon.
4. **REAL** — Mengakumulasi digit setelah titik. Jika titik tidak diikuti digit → emit intcon + period secara terpisah. Jika diikuti digit → emit realcon.
5. **NEGATIVE** — Jika - diikuti digit → lanjut ke NUM sehingga menghasilkan intcon bernilai negatif. Jika tidak → emit minus.
6. **STRING** → **ESCAPE** — Mengakumulasi isi literal. Saat menemukan ' kedua masuk ke **ESCAPE**: jika berikutnya ' lagi → itu escape character, kembali ke **STRING**; jika bukan → cek panjang currWord: 1 karakter → charcon, lebih → string.
7. **LE / GTR / COLON / EQ** — State lookahead satu karakter untuk operator dua karakter (<=, <>, >=, :=, ==). Jika karakter berikutnya tidak cocok, emit operator satu karakter.
8. **OPCMT / CURLCMNT / COMMENTS / CLSCMT** — Menangani dua gaya komentar: { } dan (* *). Seluruh isi komentar diakumulasi lalu di-emit sebagai token comment.

BAB III

PENGUJIAN

1. Test Case 1 – Normal Case

Input : input1.txt

```
program Hello;

var
  a, b: integer;

begin
  c := 12
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.
```

Output : out1.txt

test > M1 > out1.txt	16 becomes
1 programsy	17 intcon(5)
2 ident(Hello)	18 semicolon
3 semicolon	19 ident(b)
4 varsy	20 becomes
5 ident(a)	21 ident(a)
6 comma	22 plus
7 ident(b)	23 intcon(10)
8 colon	24 semicolon
9 ident(integer)	25 ident(writeln)
10 semicolon	26 lparent
11 beginsy	27 string(Result =)
12 ident(c)	28 comma
13 becomes	29 ident(b)
14 intcon(12)	30 rparent
15 ident(a)	31 semicolon
16 becomes	32 endsy
17 intcon(5)	33 period

2. Test Case 2 – Integer and Real cases

Input : input2.txt

```
123 -45 3.14 -2.71 5. 0.99 .5 ..6..7 6.7.6.7.6
```

Output : out2.txt

```

test > M1 > ≡ out2.txt
1  intcon(123)
2  intcon(-45)
3  realcon(3.14)
4  realcon(-2.71)
5  intcon(5)
6  period
7  realcon(0.99)
8  period
9  intcon(5)
10 period
11 period
12 intcon(6)
13 period
14 period
15 intcon(7)
16 realcon(6.7)
17 period
18 realcon(6.7)
19 period
20 intcon(6)

```

3. Test Case 3 – Comments and Case-Insensitive Cases

Input : input3.txt

```

{ brace comment } (* paren comment *)
BEGIN end If THEN while FOR
{ mixed comment *) (* six seven }
(* inline { (* sevens sixx }

```

Output : out3.txt

```

test > M1 > ≡ out3.txt
1  comment( brace comment )
2  comment( paren comment )
3  beginsy
4  endsy
5  ifsy
6  thensy
7  whilesy
8  forsy
9  comment( mixed comment )
10 comment( six seven )
11 comment( inline { (* sevens sixx )

```

4. Test Case 4 – All Operators Cases

Input : input4.txt

```
a := b + c - d * e / f
x <> y x <= y x >= y x < y x > y
( ) [ ] , ; :
```

Output : out4.txt

test > M1 > ≡ out4.txt	17 ident(y)
1 ident(a)	18 ident(x)
2 becomes	19 geq
3 ident(b)	20 ident(y)
4 plus	21 ident(x)
5 ident(c)	22 lss
6 minus	23 ident(y)
7 ident(d)	24 ident(x)
8 times	25 gtr
9 ident(e)	26 ident(y)
10 rdiv	27 lparent
11 ident(f)	28 rparent
12 ident(x)	29 lbrack
13 neq	30 rbrack
14 ident(y)	31 comma
15 ident(x)	32 semicolon
16 leq	33 colon

5. Test Case 5 – Unknown, Char, and String Cases

Input : input5.txt

```
'A' 'hello' 'it's'
= @#$^&*!?\|
'unclosed
a + b + -b - -5
```

Output : out4.txt

test > M1 > ≡ out5.txt	11 unknown(!)
1 charcon(A)	12 unknown(?)
2 string(hello)	13 unknown()
3 string(it's)	14 unknown(\)
4 unknown(=)	15 unknown('unclosed')
5 unknown(@)	16 ident(a)
6 unknown(#)	17 plus
7 unknown(\$)	18 ident(b)
8 unknown(^)	19 plus
9 unknown(&)	20 minus
10 times	21 ident(b)
11 unknown(!)	22 minus
12 unknown(?)	23 intcon(-5)

BAB IV

KESIMPULAN DAN SARAN

1. Kesimpulan

Pengerjaan tugas besar Milestone 1 ini menghasilkan sebuah lexer dari kompiler Arion yang akan dibuat di tugas besar ini. Pengerjaan ini dibuat dengan mendesain DFA dan implementasinya dalam bahasa C++. Pengerjaan ini memberikan pemahaman mendalam mengenai implementasi DFA secara praktikal juga bagaimana cara mentranslasikan ke dalam sebuah bahasa pemrograman.

Pengujian dari beberapa *test case* yang sudah ada memberikan hasil bahwa lexer ini sudah berjalan dengan baik dari rencana awal. Sehingga, pengerjaan kompiler ini dapat dilanjutkan pada milestone berikutnya.

2. Saran

Beberapa saran yang bisa diberikan selama pengerjaan ini adalah:

Kepada Kelompok:

- Kelompok sebaiknya menerapkan manajemen waktu yang lebih baik agar tidak mengerjakan tugas mendekati waktu tenggat
- Kelompok sebaiknya dapat melakukan planning yang lebih matang, sehingga pengerjaan seperti DFA dan implementasinya dapat dilakukan secara lebih runut

Kepada Asisten

- Asisten diharapkan dapat memperjelas lebih jauh spesifikasi pada tugas besar, sehingga tidak akan ada terjadi kebingungan atau QnA yang menumpuk

LAMPIRAN

Link Github

<https://github.com/Nusss0/CNA-Tubes-IF2224-2026>

Link Workspace Diagram Transisi DFA

https://drive.google.com/file/d/1HnY-_xOSDdAnePJDUrMultWSP0Y3I7oP/view?usp=sharing

Pembagian Tugas

13524020	25%
13524080	25%
13524104	25%
13524106	25%

REFERENSI

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Boston, MA: Addison-Wesley. [Scientific Research Publishing](#).